AXLERO SOLUTIONS

OmniBrain

Agentic Multi-Modal RAG Orchestrator

Advanced Data Science & ML Engineering

Real-World Applications - Hands-on System Building

Project	Project 1 - OmniBrain
Domain	Large Language Models (LLMs) and Agentic AI
Report purpose	Technical implementation, validation, and portfolio review
Prepared	17 August 2026

A technical report on an enterprise-style multi-agent retrieval system for evidence-grounded research.

Submitted By-

Laukik Suryavanshi

Rakhi Garhwal

Nishant Saurav

Swapnil Kalita

Rudraksh Pandey

Executive Summary

OmniBrain is an agentic, multi-modal Retrieval-Augmented Generation (RAG) system designed for research tasks that require evidence from both unstructured documents and structured market data. The project addresses a central limitation of conventional RAG: a single retrieval path cannot reliably answer questions involving PDF prose, financial tables, charts, historical price records, and multi-step reasoning.

The implemented solution uses a LangGraph-style supervisor workflow to route requests to specialist agents. Document questions are directed to semantic retrieval in Qdrant, while unambiguous historical-price questions are directed to a parameterized SQLite market-data service. The system also implements a Self-RAG-inspired correction loop: low-confidence document retrieval is retried with a narrowed search query; if evidence remains weak, the system refuses to provide an ungrounded answer.

Portfolio outcome: The project demonstrates agent routing, vector retrieval, structured-data querying, evidence citations, deterministic evaluation, and guarded failure behavior in one practical research workspace.

1. Problem Statement and Use Case

Standard RAG pipelines are often effective for plain text but struggle when a corporate report contains tables, charts, images, and references that must be combined with data from a separate financial database. OmniBrain is designed as an orchestrator rather than a single chatbot: it chooses the correct information source and specialist reasoning path for each prompt.

Representative quantitative-analyst workflow

- A researcher uploads a large corporate financial PDF and asks an evidence-based question.
- The system parses and indexes document chunks for semantic retrieval, preserving filename and page metadata for citations.
- For report-specific questions, the supervisor selects document retrieval and returns cited evidence.
- For a question such as 'What was AAPL's closing price on 2025-01-10?', the supervisor selects the SQL route and queries the market-data table.
- For weak document matches, the system retries using a rewritten, narrower query before declining to answer without evidence.

2. Solution Architecture

OmniBrain is organised as a service-oriented, agentic application. The supervisor receives the user request and selects an appropriate specialist route. The application keeps document retrieval, structured-data lookup, authentication/metadata, object storage, and output validation as separate concerns.

Layer	Role	Key technologies / implementation
User workspace	Upload PDFs, ask questions, inspect responses and sources.	Streamlit research workspace
Orchestration	Maintain state and choose a specialist agent.	LangGraph-style OmniBrainGraph and supervisor routing
Document retrieval	Find relevant chunks from uploaded PDFs.	Qdrant vector search; RAG service
Structured data	Answer market-history questions with deterministic data access.	SQLite market_prices table; parameterized queries
Guardrails	Retry weak retrieval; require citations; refuse unsupported answers.	RetrievalGuardrail and Self-RAG loop
Platform services	Store files and metadata in a deployable environment.	MinIO, PostgreSQL, Qdrant, FastAPI, Docker Compose

Request flow

Step	System action	Evidence / control
1	User submits a question with optional document context.	Question, user ID, and document ID are carried in agent state.
2	Supervisor classifies the information need.	Deterministic pre-routing handles clear market-data prompts.
3A	Document route retrieves semantic chunks from Qdrant.	Scores and source metadata are retained.
3B	SQL route queries historical market data.	Application-defined, parameterized query; no arbitrary user SQL.
4	Weak retrieval is retried once with a refined query.	Minimum relevance threshold = 0.45.
5	Response is validated for grounding and citations.	Uncited/weak answers are blocked or refused.

3. Key Modules

3.1 Agentic Orchestrator (LangGraph)

The orchestrator coordinates agent state, routing, and final response assembly. The project includes document and SQL agents, with a supervisor layer that can distinguish uploaded-document prompts from historical-market questions. This provides a clear audit trail for why a particular information source was selected.

3.2 Multi-Modal Retrieval (Qdrant / FAISS concept)

Qdrant is used as the semantic vector store for document chunks. The intended multi-modal design supports textual content today and defines an extension point for image embeddings, such as CLIP, when charts and embedded figures are indexed. Qdrant provides similarity search; source page and filename metadata make retrieved evidence understandable to the user.

3.3 Vision-Language Model (VLM) capability

The target design includes a Vision-Language Model such as GPT-4o or LLaVA to extract and reason over tables, charts, and images embedded in financial PDFs. This is an advanced roadmap module: the current implementation focuses on parsed document text and agent routing, while the architecture leaves a specialist vision-agent route for future chart/table understanding.

3.4 Text-to-SQL / Structured Market Data

To demonstrate structured-data reasoning safely, OmniBrain uses a local SQLite database containing historical AAPL and MSFT records. The SQL agent is selected for price, OHLC, and volume questions. It uses application-defined, parameterized queries rather than executing free-form SQL supplied by a user, reducing the risk of SQL injection and unpredictable database modification.

4. Implemented Reasoning Audit

A Mid-Project Review reasoning audit was implemented to prove the supervisor's routing behavior. The audit separates two classes of prompts: document evidence requests and structured market-data requests.

Prompt	Expected route	Expected data source
Summarize the uploaded annual report.	document_agent	Qdrant vector retrieval
What does the revenue table in this PDF show?	document_agent	Qdrant vector retrieval
What was AAPL's closing price on 2025-01-10?	sql_agent	SQLite market_prices
Show MSFT trading volume on 2025-01-10.	sql_agent	SQLite market_prices

Audit rationale: The deterministic pre-router makes unambiguous market-data decisions repeatable, so a selected PDF cannot accidentally override an explicitly structured time-series request.

5. Self-Correction Loop and Guardrails

The Self-RAG-inspired loop is the project's primary hallucination-control mechanism. The first document retrieval result is evaluated against a relevance threshold. If the result set is empty or its best score is below 0.45, the system rewrites the question to focus on directly stated facts, named entities, and numerical evidence, then performs one additional retrieval attempt.

Condition	System response	Why it matters
Strong retrieval	Use retrieved context to generate a cited answer.	Keeps answers tied to source material.
Weak first retrieval	Rewrite query and retrieve again.	Corrects vague or overly broad searching.
Weak second retrieval	Return a transparent refusal.	Prevents fabricated answers.
Missing citations	Block the generated response and request a more specific question.	Enforces traceability.

Automated validation

The self-correction audit contains four automated checks: (1) weak retrieval requires a retry, (2) a rewritten query is narrower than the original, (3) an uncited answer is blocked, and (4) a cited answer is accepted. The command used is shown below.

```
$env:PYTHONPATH = "backend"
python -m unittest backend.tests.test_self_rag_guardrails -v
```

```
PS C:\Users\user\OneDrive\Desktop\Axlero Internship\OmniBrain> $env:PYTHO
NPATH = "backend"
PS C:\Users\user\OneDrive\Desktop\Axlero Internship\OmniBrain> python -m
unittest backend.tests.test_self_rag_guardrails -v
test_cited_answer_passes (backend.tests.test_self_rag_guardrails.SelfRagG
uardrailAudit.test_cited_answer_passes) ... ok
test_rewritten_query_is_narrower_than_original (backend.tests.test_self_r
ag_guardrails.SelfRagGuardrailAudit.test_rewritten_query_is_narrower_than
_original) ... ok
test_uncited_answer_is_blocked (backend.tests.test_self_rag_guardrails.Se
```

Figure 1. Self-RAG guardrail tests passing in PowerShell.

6. Evaluation, Observability, and Future Guardrails

The current implementation provides deterministic unit tests, retrieval scoring, source metadata, citation validation, and transparent failure responses. For enterprise deployment, the evaluation and policy layer can be extended with Langfuse for traces, quality metrics, and prompt evaluations, and NVIDIA NeMo Guardrails for policy-driven controls such as topic restrictions, unsafe-content filtering, and structured output constraints.

Control area	Current project approach	Enterprise extension
Grounding	Relevance threshold, retry, citations, refusal.	Retrieval faithfulness and answer-correctness metrics.
Routing	Deterministic audit tests for document vs SQL prompts.	Route-level dashboards and drift monitoring.
Safety	No arbitrary SQL; response refusal on insufficient evidence.	NeMo Guardrails policies and red-team evaluation.
Observability	Logs and test evidence.	Langfuse traces, prompt/version analytics, feedback loops.

7. Technology Stack

Category	Technology	Purpose
Backend API	FastAPI + Uvicorn	HTTP routes for upload, chat, health, and authentication.
Agent workflow	LangGraph / LangChain ecosystem	Stateful specialist-agent orchestration.

Category	Technology	Purpose
LLM provider	Groq-compatible chat model configuration	Fast inference for supervisor and answer generation.
Vector store	Qdrant	Semantic search over indexed document chunks.
Market-data demo	SQLite	Safe, reproducible historical-price lookup.
Application metadata	PostgreSQL	Users, document metadata, and transactional application state.
File storage	MinIO	Object storage for uploaded documents.
Frontend	Streamlit	Research workspace interface.
Deployment	Docker Compose	Reproducible local services.

8. Testing and Evidence

The project includes automated tests for both supervisor routing and Self-RAG guardrails. These tests are intentionally deterministic: they can be run without relying on a live model response, making them useful as review evidence and regression protection.

- Routing audit: verifies document questions choose the document agent and market questions choose the SQL agent.
- Self-RAG audit: verifies retry behavior, query rewriting, citation blocking, and citation acceptance.
- Safe SQL design: market queries use application-defined and parameterized access patterns.
- Documented test output: the report includes a screenshot of the passing self-RAG suite.

```
$env:PYTHONPATH = "backend"
python -m unittest discover -s backend/tests -v
```

9. Local Execution Guide

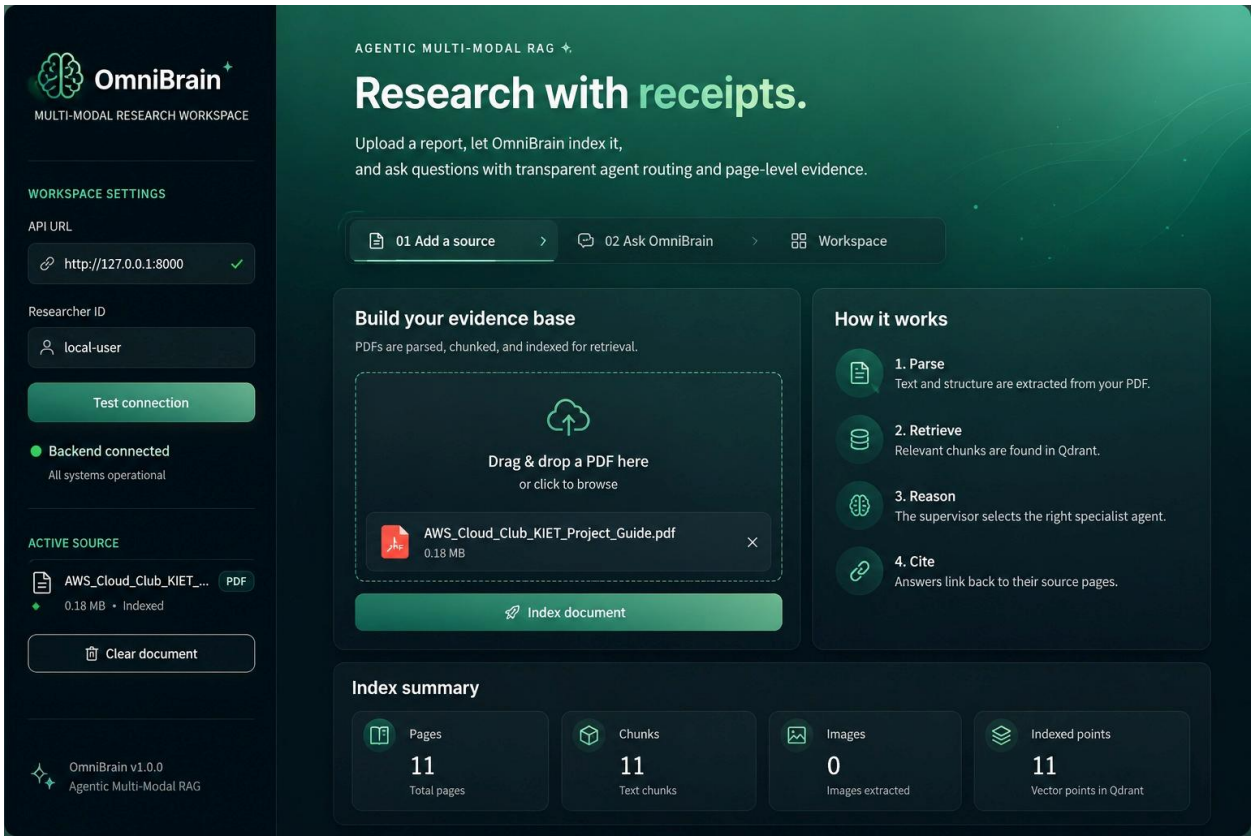
The full environment depends on Docker Desktop because the backend connects to PostgreSQL, MinIO, and Qdrant. Secrets must be stored in a local .env file and must never be committed to GitHub.

Step	Command / action
1. Start services	docker compose up -d --build
2. Verify API	Open http://localhost:8000/docs

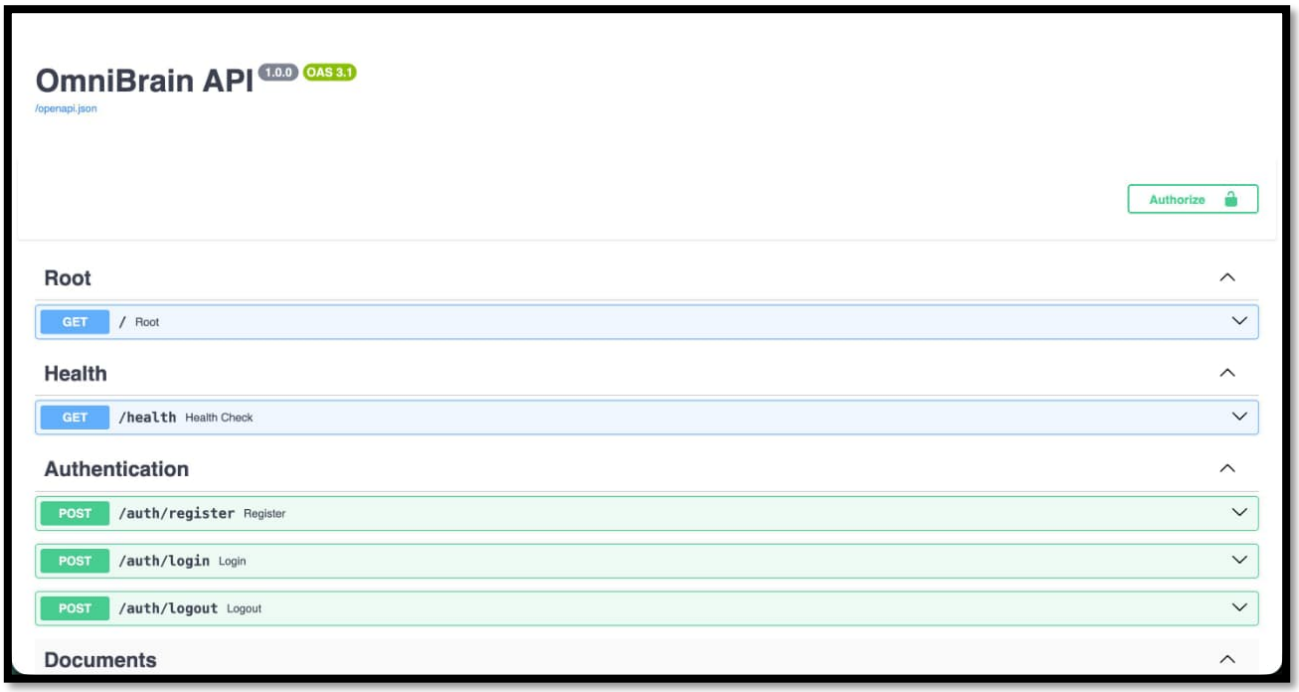
Step	Command / action
3. Start workspace	python -m streamlit run frontend\app.py
4. Open UI	Open the Streamlit URL, normally http://localhost:8501
5. Run tests	Set PYTHONPATH=backend and run unittest as shown above.

Configuration note: The local .env file requires LLM credentials plus PostgreSQL, MinIO, Qdrant, and authentication settings. These are environment secrets and are excluded from version control.

Frontend Design



Backend Design


OmniBrain API
1.0.0
OAS 3.1
/openapi.json
Authorize
Root
GET / Root
Health
GET /health Health Check
Authentication
POST /auth/register Register
POST /auth/login Login
POST /auth/logout Logout
Documents


Root
GET / Root
Health
GET /health Health Check
Authentication
POST /auth/register Register
POST /auth/login Login
POST /auth/logout Logout
Documents
POST /upload Upload Document
DELETE /upload/{document_id} Delete Document
PUT /upload/{document_id}/owner Assign Document Owner
Document Chat
POST /chat Chat With Document

10. Challenges, Limitations, and Next Steps

- Infrastructure complexity: the complete system requires several services; Docker Compose is the recommended local-run path.
- Vision reasoning is an architectural target; adding a dedicated VLM route and image embeddings is the next major multi-modal milestone.
- The current market-data table is intentionally small and reproducible; production use should connect to a governed market-data provider and schema.
- The current guardrail layer is deterministic and transparent; Langfuse and NeMo Guardrails can strengthen observability, policy enforcement, and evaluation at scale.
- Evaluation can be extended with a labelled question set, citation-faithfulness scoring, route-accuracy metrics, and adversarial prompts.

Conclusion

OmniBrain demonstrates a practical path from a standard document chatbot to an enterprise-oriented agentic research system. Its most important contribution is not simply retrieval, but controlled decision-making: the system can select document search or structured SQL retrieval, retry failed retrieval responsibly, surface citations, and decline unsupported answers. The implemented audit tests and documented evidence make the project suitable for a senior-level data science and ML engineering portfolio review.

References and Evidence

- Axlero Solutions project brief: 'OmniBrain: Agentic Multi-Modal RAG Orchestrator.'
- Repository documentation: docs/midproject_reasoning_audit.md.
- Automated tests: backend/tests/test_routing_audit.py and backend/tests/test_self_rag_guardrails.py.
- Implementation modules: backend/app/agents/routing.py, backend/app/agents/sql_agent.py, backend/app/services/market_data_service.py, backend/app/services/guardrail_service.py, and backend/app/services/rag_service.py.

Project Link

https://github.com/laukiksuryavanshi17/OmniBrain